

Génération et suivi de trajectoire par une voiture autonome

-

Programmes Python

LÉRONDEL-TOUZÉ Adam, TIPE, Candidat n°23764

TABLE DES MATIÈRES

1	1 ^{ères} lignes peu ou prou communes	3
2	Affichage des tentacules en arc de cercle	5
3	Affichage des tentacules en forme de clothoïdes	8
4	Dernières lignes peu ou prou communes	10
5	Déplacement de la voiture sur une tentacule, sans tenir compte des obstacles	12
6	Caractérisation de la navigabilité d'une tentacule	17
7	Déplacement en évitant les obstacles (sans choix optimal de tentacule)	20
8	Critère de choix d'une tentacule	22
9	Optimisation de l'algorithme et ajout de "l'houache" de la "voiture"	24
10	Améliorations - Code final	27

1ÈRES LIGNES PEU OU PROU COMMUNES

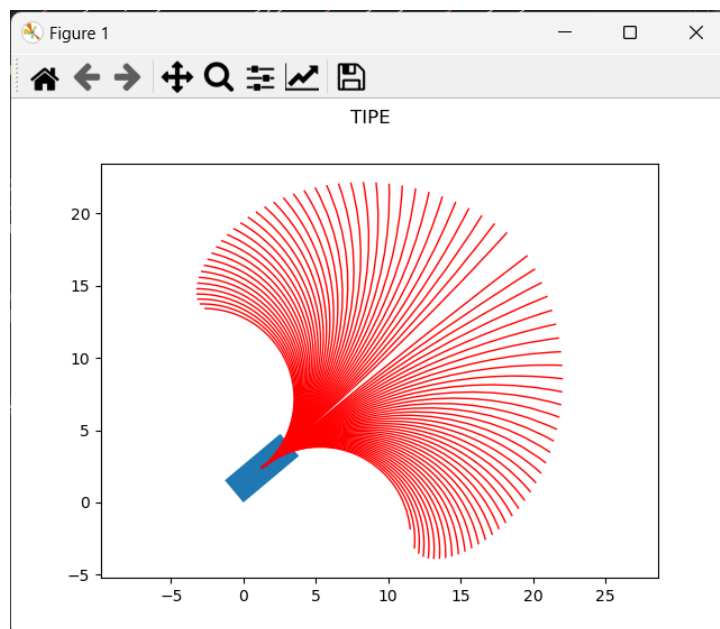
```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Dec 19 19:50:23 2022
4
5  @author: adaml
6  """
7
8
9  import numpy as np
10 import matplotlib.pyplot as plt
11 import matplotlib.animation
12 import matplotlib.patches
13 import matplotlib.transforms
14 import matplotlib.lines
15 import scipy.interpolate
16 import shapely
17 import shapely.geometry
18 import time
19 from scipy.integrate import quad as integrale
20
21
22 VITESSE_INIT_VOIT = 5 # vitesse initiale du véhicule
23 VITESSE_MAX_VOIT = 15 # vitesse maximale (pour rester dans les zones linéaires des modèles)
24 ACCEL_LAT_MAX = 1.5 # accélération latérale maximale (pour rester dans les zones linéaires
    ↪ des modèles)
25 LONGUEUR_VEHIC = 5 # longueur du véhicule
26 LARGEUR_VEHIC = 2 # largeur du véhicule
27 POSITION_VOITURE_INIT = (-10, -2) # position (x, y) initiale de la voiture
28 INCLINAISON_VOITURE_INIT = 0 # inclinaison initiale de la voiture (en degrés)
29 BRAQUAGE_VOLANT_INIT = 0 # angle volant/voiture en degrés
30 PRECISION = 15 # nombre de points définissant l'arc de cercle. ie nombre de positions où la
    ↪ voiture ira
31 TEMPS_REAC = 90 # temps de génération de tentacules
32 CONE_BRAQUAGE = 0.02 # valeur de braquage définissant des trajectoires "lisses". cf
    ↪ get_tentacules_navigables_et_lisses
33 COMMENTAIRES = True # si des commentaires sur l'évolution de l'algo doivent être affichés
    ↪ dans la console
34 PETIT_POUCKET = True # si on veut récupérer la trajectoire de la voiture
35 SECURITE = True # vérifie les collisions. False si aucune sécu, None si check sans arrêter,
    ↪ True si arrêt lorsque collision
36
37 # courbure = ro
38 # courbure = 1/rayon dans le cas d'un cercle
39 # braquage = delta = arctan(L/rayon) quasi logique (schéma + perpendiculaire)
40 # longueur = angle*rayon

```

```
41 # empattement += longueur voiture
42
43
44 """
45 SOME CODE
46 """
47
```

AFFICHAGE DES TENTACULES EN ARC DE CERCLE



```

1  """
2  Affiche la voiture avec ses tentacles en forme d'arcs de cercle.
3  """
4
5
6  class Tentacule:
7      def __init__(self, voiture, k, delta_k, axes):
8          self.axes = axes
9
10         self.L1 = 8 + 72 * (voiture.vitesse/voiture.vitesse_max)**1.2 # formule empirique,
            ↳ équation (9.5) p.123

```

```

11     # formules pour la longueur p.123 équation (9.4)
12     # formules pour le rayon de courbure p.122 équation (9.3)
13
14     if 1 <= k <= 40: # tentacule à orienter vers la droite
15         self.longueur = self.L1 + 20 * np.sqrt((k-1) / 40)
16         self.courbure = -np.tan(delta_k) / voiture.longueur
17
18         self.rayon = 1 / self.courbure
19         self.angle_rad = self.longueur / self.rayon
20         self.angle = self.angle_rad * 180 / np.pi
21
22         self.patch = matplotlib.patches.Arc((voiture.x + self.rayon / 2, voiture.y),
23         ↪ self.rayon, self.rayon, theta1=180 - self.angle, theta2=180, color='r')
24
25     elif k == 41:
26         self.longueur = self.L1 + 20 * np.sqrt((k-1) / 40)
27         self.courbure = 0
28
29         self.patch = matplotlib.patches.Arc((0, 0), 0, 0)
30
31     else: # 42<=k<=81 # tentacule à orienter vers la gauche
32         self.longueur = self.L1 + 20 * np.sqrt((81-k) / 40)
33         self.courbure = np.tan(delta_k) / voiture.longueur
34
35         self.rayon = 1 / self.courbure
36         self.angle_rad = self.longueur / self.rayon # en radian
37         self.angle = self.angle_rad * 180 / np.pi # en degrés
38
39         self.patch = matplotlib.patches.Arc((voiture.x - self.rayon / 2, voiture.y),
40         ↪ self.rayon, self.rayon, theta1=0, theta2=self.angle, color='r')
41
42     self.patch.set_transform(
43     ↪ matplotlib.transforms.Affine2D().rotate_deg_around(voiture.x, voiture.y,
44     ↪ voiture.angle - 90) + self.axes.transData)
45
46 class Voiture:
47     def __init__(self, shape_voiture, x, y, axes):
48         self.axes = axes
49         self.longueur = LONGUEUR_VEHIC
50         self.largeur = LARGEUR_VEHIC
51         self.vitesse_max = VITESSE_MAX_VOIT
52         self.accel_lat_max = ACCEL_LAT_MAX
53         self.shape_voiture = shape_voiture
54
55         self.vitesse = VITESSE_INIT_VOIT
56         self.x, self.y = self.shape_voiture.get_center()
57         self.angle = INCLINAISON_VOITURE_INIT
58
59         self.courbure_max = self.accel_lat_max / self.vitesse ** 2 # formule p.122 équation
60         ↪ (9.1)
61         self.braquage_max = np.arctan(self.longueur * self.courbure_max) # formule p.122
62         ↪ équation (9.2)
63
64         self.transfo_init = self.shape_voiture.get_transform() +
65         ↪ matplotlib.transforms.Affine2D().rotate_deg(90)
66
67 figure_ref_terre, axes_ref_terre = plt.subplots()
68 figure_ref_terre.suptitle("TIPE")
69 simulation_voision_terrestre = Simulation_Ref_Terre()
70

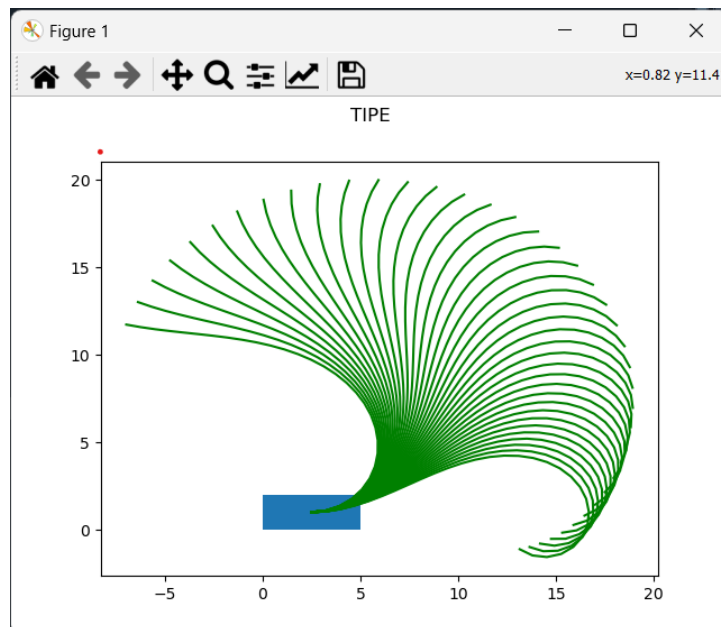
```

```

66 shape_voiture = matplotlib.patches.Rectangle(POSITION_VOITURE_INIT, LONGUEUR_VEHIC,
↪  LARGEUR_VEHIC, INCLINAISON_VOITURE_INIT)
67 voiture = Voiture(shape_voiture, 0, 0, axes_ref_terre)
68 axes_ref_terre.add_patch(voiture.shape_voiture)
69 tentacules = []
70 deltas = np.linspace(-voiture.braquage_max, voiture.braquage_max, 80)
71 for k in range(1, 81):
72     tentacules.append(Tentacule(voiture, k, deltas[k-1], axes_ref_terre))
73 for k in range(80):
74     axes_ref_terre.add_patch(tentacules[k].patch)
75
76
77 plt.autoscale()
78 plt.axis('equal')
79 plt.show()
80

```

AFFICHAGE DES TENTACULES EN FORME DE CLOTHOÏDES



```

1  """Affiche la voiture avec ses tentacules clothoïdaires.
2  """
3
4
5  def y(t, rap, ro):
6      return integrale(lambda p: np.cos(rap*p**2 + ro*p)/3, 0, t)[0]
7
8
9  def x(t, rap, ro):
10     return integrale(lambda p: np.sin(rap*p**2 + ro*p)/3, 0, t)[0]
11

```

```

12
13 courbure_tentac_initiale = BRAQUAGE_VOLANT * np.pi / 180
14
15
16 class Tentacule:
17     def __init__(self, voiture, rap, ro, axes, L):
18         self.voiture = voiture
19         self.axes = axes
20
21         self.L = L # formule empirique, équation (9.15) p.128
22         self.precision = PRECISION
23
24         xdata = [x(t, rap, ro) for t in np.linspace(0, int(L), self.precision)]
25         ydata = [y(t, rap, ro) for t in np.linspace(0, int(L), self.precision)]
26         self.patch = matplotlib.lines.Line2D(xdata, ydata, color='green')
27         self.patch.set_transform(matplotlib.transforms.Affine2D().rotate_deg_around(0, 0,
28             ↪ voiture.angle - 90) + matplotlib.transforms.Affine2D().translate(voiture.x,
29             ↪ voiture.y) + self.axes.transData)
30
31 figure_ref_terre, axes_ref_terre = plt.subplots()
32 figure_ref_terre.suptitle("TIPE")
33 simulation_voision_terrestre = Simulation_Ref_Terre()
34
35 shape_voiture = matplotlib.patches.Rectangle(POSITION_VOITURE_INIT, LONGUEUR_VEHIC,
36     ↪ LARGEUR_VEHIC, INCLINAISON_VOITURE_INIT)
37 voiture = Voiture(shape_voiture, 0, 0, axes_ref_terre)
38 axes_ref_terre.add_patch(voiture.shape_voiture)
39 tentacules = []
40
41 L = 7*VITESSE_INIT_VOIT - 5 # formule empirique, équation (9.15) p.128
42 ro_max = ACCEL_LAT_MAX / VITESSE_INIT_VOIT**2
43 distance_collision = VITESSE_INIT_VOIT**2 / 2*ACCEL_LAT_MAX
44
45 rap_min = (-ro_max - courbure_tentac_initiale) / (distance_collision/2)
46 rap_max = (+ro_max - courbure_tentac_initiale) / (distance_collision/2)
47 deltas = np.linspace(rap_min, rap_max, 40)
48 for rap in deltas:
49     tentacules.append(Tentacule(voiture, rap, courbure_tentac_initiale + rap*L,
50         ↪ axes_ref_terre, L))
51 for k in range(40):
52     axes_ref_terre.add_line(tentacules[k].patch)
53
54 plt.autoscale()
55 plt.show()
56

```

DERNIÈRES LIGNES PEU OU PROU COMMUNES

```

1  """
2  SOME CODE
3  """
4
5
6  # création figure + axes
7  figure_ref_terre, axes_simu = plt.subplots()
8  figure_data, (axes_vitesse, axes_braquage) = plt.subplots(2, 1, sharex=True)
9  # titre
10 figure_ref_terre.suptitle("Animation")
11 figure_data.suptitle("Données en temps réel")
12 axes_vitesse.set_title("Vitesse")
13 axes_braquage.set_title("Braquage")
14 axes_braquage.set_xlabel("Temps")
15
16 # création de la forme de la voiture
17 shape_voiture = matplotlib.patches.Rectangle(POSITION_VOITURE_INIT, LONGUEUR_VEHIC,
18 ↪ LARGEUR_VEHIC, fill=False, zorder=float('inf'))
19 shape_voiture.rotation_point = 'center'
20 shape_voiture.set_angle(INCLINAISON_VOITURE_INIT)
21
22 # création de la voiture
23 voiture = Voiture(shape_voiture, [axes_simu, axes_vitesse, axes_braquage],
24 ↪ Tentacule_clothoïdaire)
25
26 # ajout du dessin de la voiture à la figure
27 axes_simu.add_patch(voiture.shape_voiture)
28
29 # création de la forme de l'obstacle
30 shape_obstacle = matplotlib.patches.Rectangle((30, -2.5), 2, 1.5, fill=True,
31 ↪ zorder=float('inf'), color='cyan')
32 obstacle1 = Obstacle_fixe(shape_obstacle)
33 axes_simu.add_patch(obstacle1.patch)
34
35 # création des bords de route
36 shape_route_haut = matplotlib.patches.Rectangle((-10, 5), 90, 2)
37 route_haut = Obstacle_fixe(shape_route_haut)
38 axes_simu.add_patch(route_haut.patch)
39
40 shape_route_bas = matplotlib.patches.Rectangle((-10, -5), 78, 2)
41 route_bas = Obstacle_fixe(shape_route_bas)
42 axes_simu.add_patch(route_bas.patch)
43
44 shape_route_gauche = matplotlib.patches.Rectangle((68, -45), 2, 42)
45 route_gauche = Obstacle_fixe(shape_route_gauche)

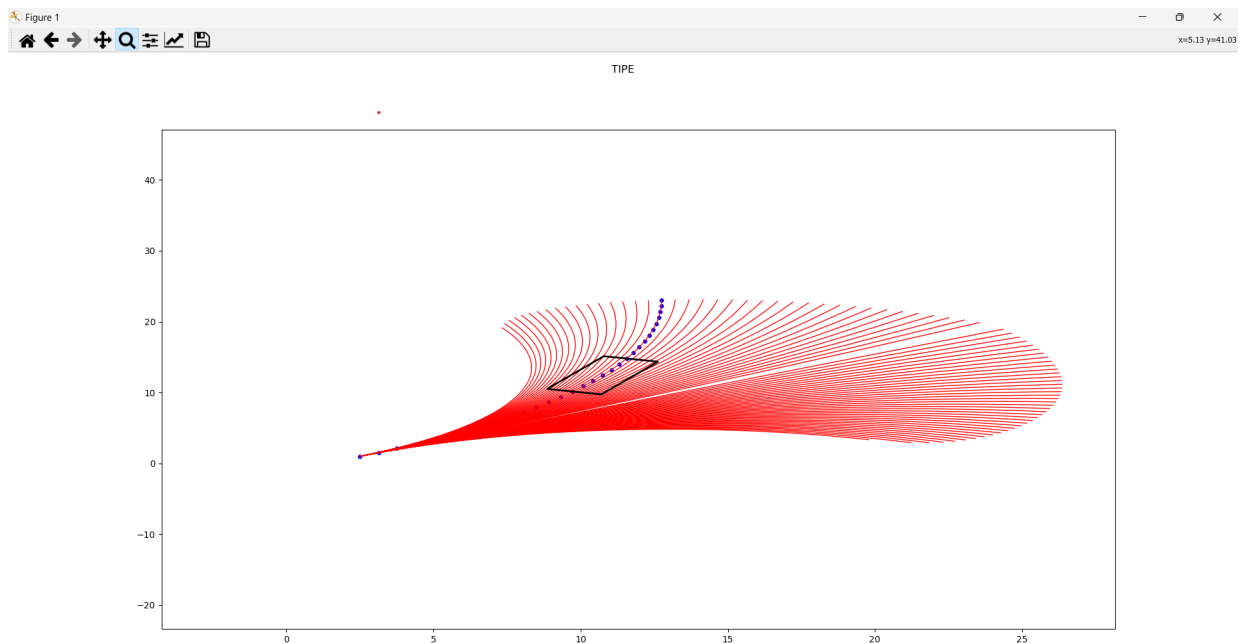
```

```

43 axes_simu.add_patch(route_gauche.patch)
44
45 shape_route_droite = matplotlib.patches.Rectangle((78, -45), 2, 50)
46 route_droite = Obstacle_fixe(shape_route_droite)
47 axes_simu.add_patch(route_droite.patch)
48
49 # création de la simulation
50 simulation = Simulation_Ref_Terre([voiture], [obstacle1, route_bas, route_haut,
↳ route_droite, route_gauche])
51
52 # création et lancement de l'animation
53 anim = matplotlib.animation.FuncAnimation(figure_ref_terre,
54                                           simulation.animer,
55                                           init_func=lambda: [voiture.shape_voiture for
↳ voiture in simulation.voitures] +
↳ [tentacule.patch for tentacule in
↳ simulation.new_tentacules] + [tentacule.patch
↳ for tentacule in simulation.old_tentacules],
56                                           fargs=[],
57                                           interval=35,
58                                           cache_frame_data=False,
59                                           blit=True,
60                                           repeat=True)
61
62 # création d'un timer
63 timer = figure_ref_terre.canvas.new_timer(interval=TEMPS_REAC)
64 if SECURITE is not False:
65     timer2 = figure_ref_terre.canvas.new_timer(40)
66     timer2.add_callback(simulation.collisions)
67     timer2.start()
68     simulation.ajout_timer(timer2)
69     figure_ref_terre.canvas.mpl_connect('close_event', lambda event: timer2.stop())
70 # ajout d'une fonction appelée itérativement
71 timer.add_callback(simulation.gestion_tentac, voiture)
72 # démarrage du timer
73 timer.start()
74
75 simulation.ajout_anim(anim)
76 simulation.ajout_timer(timer)
77 # stop le timer si on ferme la figure (pour éviter qu'une fonction continue de tourner en
↳ boucle sans intérêt une fois tout terminé, c'est plutôt mal vu...!)
78 figure_ref_terre.canvas.mpl_connect('close_event', lambda event: timer.stop())
79 figure_ref_terre.canvas.mpl_connect('key_press_event', lambda event: (print("Key Pressed"),
↳ simulation.pause_replay()))
80
81 axes_simu.autoscale()
82 # axes_simu.set(xlim=(-50, 50), ylim=(-50, 150))
83 axes_simu.axis('equal')
84 plt.show()
85

```

DÉPLACEMENT DE LA VOITURE SUR UNE TENTACULE, SANS TENIR COMPTE DES OBSTACLES



```

1  """
2  Déplace la voiture indéfiniment (selon une tentacule, puis régénération des tentacules, et
   ↳ suivi, et génération, etc).
3  """
4
5
6  class Tentacule:
7      """

```

```

8      Objet tentacule, dépendant d'une voiture, et déterminé par son ordre et son angle de
↪ braquage.
9      """
10     def get_parametric(self, arc=True):
11         self.precision = PRECISION
12         if arc is True:
13             # Définition d'un arc avec un nombre de points contrôlé
14             self.patch._path = matplotlib.patches.Path.arc(self.angle_debut, self.angle_fin,
↪ self.precision)
15             # Paramétrisation de la tentacule
16             self.chemin = (self.patch.get_transform() -
↪ axes_ref_terre.transData).transform_path(
↪ self.patch.get_path()).cleaned().vertices[:-1][::self.signe]
17         if arc is False:
18             # Interpolation du chemin
19             self.chemin_interpol = scipy.interpolate.interp1d(self.chemin[:, 0],
↪ self.chemin[:, 1])
20             lch = len(self.chemin[:, 0])
21             # Interpolation de x(t) pour faire une génération d'abscisses supplémentaires
22             points = scipy.interpolate.interp1d(range(0, (lch)*self.precision,
↪ self.precision), self.chemin[:, 0]) # va jusqu'à (lch-1)*precision inclus
23             # Génération des abscisses supplémentaires avec le points(range) et des
↪ ordonnées interpolées correspondantes avec le chemin_interpol(x)
24             self.chemin = np.array([[x, self.chemin_interpol(x)] for x in points(range(0,
↪ (lch-1)*self.precision, lch-1))])
25         return self.chemin
26
27
28     class Voiture:
29         """
30         Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
31         """
32         def deplace(self):
33             """
34             S'il existe un chemin (suite de points), déplace la voiture suivant ce chemin.
35
36             Raises
37             -----
38             erreur
39                 Le chemin à suivre n'est pas défini.
40
41             Returns
42             -----
43             None.
44
45             """
46             try:
47                 new_x = self.chemin[1][0]
48                 new_y = self.chemin[1][1]
49                 vx = new_x - self.chemin[0][0]
50                 vy = new_y - self.chemin[0][1]
51                 new_angle = np.degrees(np.arctan2(vy, vx))
52                 self.shape_voiture.set(x=new_x - self.largeur/2, y=new_y - self.longueur/2,
↪ angle=new_angle)
53                 self.x, self.y, self.angle = new_x, new_y, new_angle
54                 self.chemin = np.delete(self.chemin, 0, 0)
55             except IndexError as erreur:
56                 print('TENTACULE ÉPUISÉE, ON NE SAIT PLUS QUEL CHEMIN SUIVRE')
57                 # raise erreur
58
59         def update_chemin(self, arc=True):

```

```

60         """
61         Met à jour le chemin que la voiture doit emprunter, en définissant et paramétrant la
↪ tentacule.
62
63         Returns
64         -----
65         None.
66
67         """
68         self.remove_tentacules()
69         self.generation_tentacules()
70
71         ptch = self.tentacules[NUMERO_DE_LA_TENTACULE].patch # Récupération de la forme
72         self.precision = PRECISION
73         if arc is True:
74             # Définition d'un arc avec un nombre de points contrôlé
75             ptch._path =
↪ matplotlib.patches.Path.arc(self.tentacules[NUMERO_DE_LA_TENTACULE].angle_debut,
↪ self.tentacules[NUMERO_DE_LA_TENTACULE].angle_fin, self.precision)
76         # Paramétrisation de la tentacule
77         self.chemin = (ptch.get_transform() - axes_ref_terre.transData).transform_path(
↪ ptch.get_path()).cleaned().vertices[:-1][:
↪ :self.tentacules[NUMERO_DE_LA_TENTACULE].signe]
78         if arc is False:
79             # Interpolation du chemin
80             self.chemin_interpol = scipy.interpolate.interp1d(self.chemin[:, 0],
↪ self.chemin[:, 1])
81             lch = len(self.chemin[:, 0])
82             # Interpolation de x(t) pour faire une génération d'abscisses supplémentaires
83             points = scipy.interpolate.interp1d(range(0, (lch)*self.precision,
↪ self.precision), self.chemin[:, 0]) # va jusque (lch-1)*precision inclus
84             # Génération des abscisses supplémentaires avec le points(range) et des
↪ ordonnées interpolées correspondantes avec le chemin_interpol(x)
85             self.chemin = np.array([[x, self.chemin_interpol(x)] for x in points(range(0,
↪ (lch-1)*self.precision, lch-1))])
86         self.chemin = self.tentacules[NUMERO_DE_LA_TENTACULE].get_parametric()
87         # Affichage des points que la voiture devra suivre
88         plt.scatter(x=self.chemin[:, 0], y=self.chemin[:, 1], color='b', s=8)
89
90     def generation_tentacules(self):
91         """
92         Génère les 80 tentacules que la voiture aura la possibilité d'emprunter.
93
94         Returns
95         -----
96         None.
97
98         """
99         self.tentacules = []
100         deltas = np.linspace(-self.braquage_max, self.braquage_max, 80) # 80 est un nombre
↪ empirique de tentacules défini dans la thèse (il est bon compromis entre
↪ précision et temps de calcul)
101         for k in range(1, 81):
102             self.tentacules.append(Tentacule(self, k, deltas[k-1], self.axes)) # Ajout de
↪ la tentacule aux tentacules appartenant au véhicule
103         for k in range(80):
104             self.axes.add_patch(self.tentacules[k].patch) # Ajout de la tentacule à la
↪ figure
105
106     def remove_tentacules(self):
107         """
108         Enlève les tentacules (qui appartiennent au véhicule) de la figure.

```

```

109
110     Returns
111     -----
112     None.
113
114     """
115     for tentacule in self.tentacules:
116         tentacule.patch.set(visible=False) # car si simple remove, on perd les axes de
117         ↪ référence et c pas bon pour le blit...
118
119 class Simulation:
120     """
121     Classe qui gère tout l'aspect simulation : animation + gestion des objets.
122
123     Parameters
124     -----
125     voitures: list
126         Liste des voitures présentes
127     obstacles: list # TODO
128         liste des obstacles FIXES. Ils ne seront pas animés
129     obstacles_mvt: list # TODO
130         liste des obstacles à déplacer. Fonction développée plus tard
131     """
132
133     def __init__(self, voitures=[], obstacles_fixes=[], obstacles_mvt=[]):
134         self.voitures = voitures
135         self.obstacles_fixes = obstacles_fixes
136         self.obstacles_mvt = obstacles_mvt
137         self.new_tentacules = [] # liste des tentacules à afficher (blit)
138         self.old_tentacules = [] # liste des tentacules qui ne sont plus d'actualité, mais
139         ↪ qui doivent être supprimées (blit)
140
141         for voiture in self.voitures:
142             self.gestion_tentac(voiture) # créé les tentacules de la voiture, et créé un
143             ↪ chemin à suivre (surement inexistant jusqu'alors...)
144
145     def animer(self, i):
146         """
147         Anime la figure. S'occupe de déplacer les voitures et les obstacles.
148         Note : fait appel au "task-manager des tentacules" pour savoir quoi actualiser dans
149         ↪ la figure (blit)
150
151         Parameters
152         -----
153         i : int
154             Numéro d'appel de la fonction (elle est censée être appelée itérativement par
155             ↪ une matplotlib.animation.
156
157         Returns
158         -----
159         list of artists
160             Liste des dessins à actualiser (blit).
161
162         """
163         # Déplace les voitures
164         for voiture in self.voitures:
165             voiture.deplace()
166         # Déplace les obstacles
167         for obstacle in self.obstacles_mvt:
168             obstacle.deplace()
169         try:

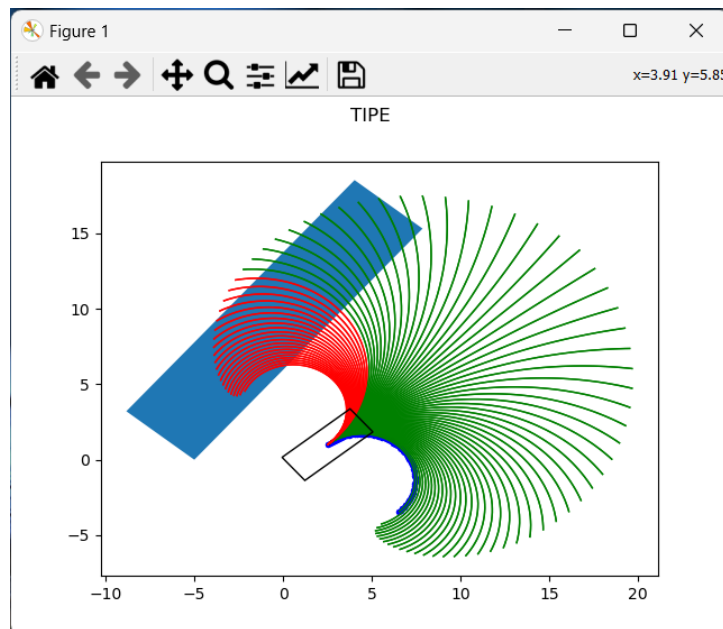
```

```

166         # Retourne la liste des dessins à actualiser dans la figure
167         # TODO : ajouter les obstacles lors de l'import de la fonctionnalité
168         return [voiture.shape_voiture for voiture in self.voitures] + [tentacule.patch
↪      for tentacule in self.new_tentacules] + [tentacule.patch for tentacule in
↪      self.old_tentacules]
169     finally:
170         # Met juste après les valeurs adéquates à jour pour ne plus avoir à redessiner
↪      des dessins fixes (blit)
171         self.old_tentacules = [] # Suppression des tentacules obsoletes, qu'on a bien
↪      effacé avec le return
172         for tentac in self.new_tentacules:
173             tentac.patch.set_animated(False) # Dés-anime les nouvelles tentacules
↪      (affichées avec le return) : elles ne doivent plus bouger jusqu'à une
↪      redéfinition de celles-ci
174         # PROBLÈME AU NIVEAU DU BLIT : IL SEMBLERAIT QU'ON SOIT OBLIGÉ DE RETOURNER
↪      TOUTES LES TENTACULES À AFFICHER, PLUTÔT QU'UNIQUEMENT CELLES À UPDATE...
175         # TODO moins urgent : résoudre ce petit problème de blit si on veut gagner un
↪      peu de temps (trace inutilement 80 tentacules...)
176         # Suppression des nouvelles tentacules de la liste des dessins à actualiser
177         # self.new_tentacules = []
178
179     def gestion_tentac(self, voiture):
180         """
181         "Task-manager des tentacules"... Gère l'ajout et la suppression des tentacules
↪      (appartenant à la voiture *voiture*) de la figure.
182
183         Parameters
184         -----
185         voiture : Voiture
186             objet voiture.
187
188         Returns
189         -----
190         None.
191
192         """
193         for tentac in voiture.tentacules: # pour mettre à jour et enlever les tentacules
↪      qui vont être supprimées
194             tentac.patch.set_animated(True) # Anime les tentacules à supprimer (elles
↪      étaient dés-animées) (blit)
195             self.old_tentacules.append(tentac) # ajout à la liste des dessins à actualiser
↪      (blit)
196
197         voiture.update_chemin() # suppression du système actuel de tentacules pour la
↪      voiture, puis génération de nouvelles tentacules pour la voiture
198
199         for tentac in voiture.tentacules: # pour mettre à jour et ajouter les tentacules
↪      qui vont être ajoutées
200             tentac.patch.set_animated(True) # anime les tentacules à ajouter (blit)
201             self.new_tentacules.append(tentac) # ajout à la liste des dessins à actualiser
↪      (blit)
202

```

CARACTÉRISATION DE LA NAVIGABILITÉ D'UNE TENTACULE



```

1  """
2  Trace les tentacules en différentes couleurs, à l'arrêt, selon qu'elles soient navigables ou
   ↳ non.
3  """
4  class Tentacule:
5      """
6      Objet tentacule, dépendant d'une voiture, et déterminé par son ordre et son angle de
   ↳ braquage.
7      """
8      def get_zone_support(self):
9          if self.voiture.vitesse < 3:

```

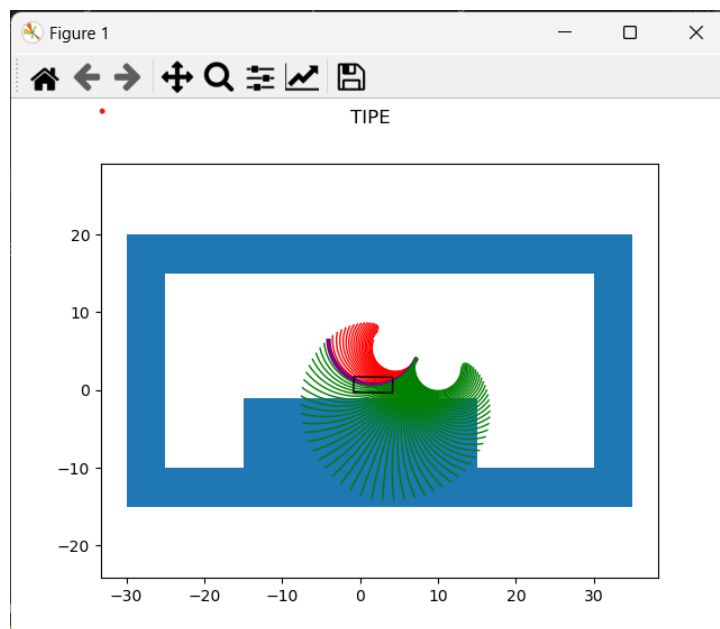
```

10         self.dist_zone_support = 1.4 + 0.2*self.voiture.vitesse/3 # formule empirique
    ↪ p.123 eq(9.6) thèse
11     elif 3 <= self.voiture.vitesse <= 15:
12         self.dist_zone_support = 1.6 + 0.6 * (self.voiture.vitesse-3) / 15 # idem
13     else:
14         raise ValueError("La vitesse est trop élevée !")
15     self.zone_support = shapely.geometry.LineString(
    ↪ self.get_parametric()).buffer(self.dist_zone_support)
16     return self.zone_support
17
18     def is_navigable(self, objets_alentours):
19         self.zone_to_be_clean =
    ↪ self.get_zone_support().intersection(self.voiture.get_espace_collision())
20     for objet in objets_alentours:
21         if objet == self.voiture: # Pas grave si la tentacule cogne sa propre voiture
22             pass
23         elif self.zone_to_be_clean.intersection(objet.shapely_poly).is_empty is False:
24             self.patch.set_color("red")
25             return False
26     return True
27
28
29 class Voiture:
30     """
31     Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
    ↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
32     """
33     def get_espace_collision(self):
34         self.distance_collision = self.vitesse**2 / 2*self.accel_lat_max
35         self.espace_collision = shapely.geometry.Point((self.x,
    ↪ self.y)).buffer(self.distance_collision + self.longueur/2)
36         return self.espace_collision
37
38     def is_tentacule_navigable(self, tentacule, objets_alentours):
39         return tentacule.is_navigable(objets_alentours)
40
41     def choix_tentacule(self, tentacules_navigables):
42         try:
43             # On prend la 1ere tentacule lisse qu'on trouve
44             for tentacule in tentacules_navigables:
45                 if tentacule.lisse is True:
46                     return tentacule
47             # Si aucune n'est lisse, on prend la 1ère
48             return tentacules_navigables[0]
49         except IndexError:
50             raise "AUCUNE TENTACULE NAVIGABLE"
51
52     def get_tentacules_navigables(self, objets_alentours):
53         tentacules_navigables = []
54         for tentacule in self.tentacules:
55             if tentacule.is_navigable(objets_alentours):
56                 tentacules_navigables.append(tentacule)
57         return tentacules_navigables
58
59     def _simu(self, simu):
60         self.simu = simu
61
62
63 class Obstacle():
64     def __init__(self, patch):
65         self.patch = patch
66         self.x, self.y = self.patch.get_center()

```

```
67         self.shapely_poly = shapely.geometry.Polygon(  
68             (self.patch.get_transform()).transform_path(self.patch.get_path()).vertices  
69         )  
70  
71     class Obstacle_fixe(Obstacle):  
72         def __init__(self, patch):  
73             super().__init__(patch)  
74
```

DÉPLACEMENT EN ÉVITANT LES OBSTACLES (SANS CHOIX OPTIMAL DE TENTACULE)



```

1  """
2  Déplace la voiture dans une enceinte carrée.
3  """
4  class Voiture:
5      """
6      Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
7      ↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
8      """
9      def choix_tentacule(self, tentacules_navigables):
10         # TODO : faire un critère de dégagement du tentacule (p. 125 section 9.2.4.1)
11         # TODO : faire un critère de changement de braquage (p.125 section 9.2.4.2)

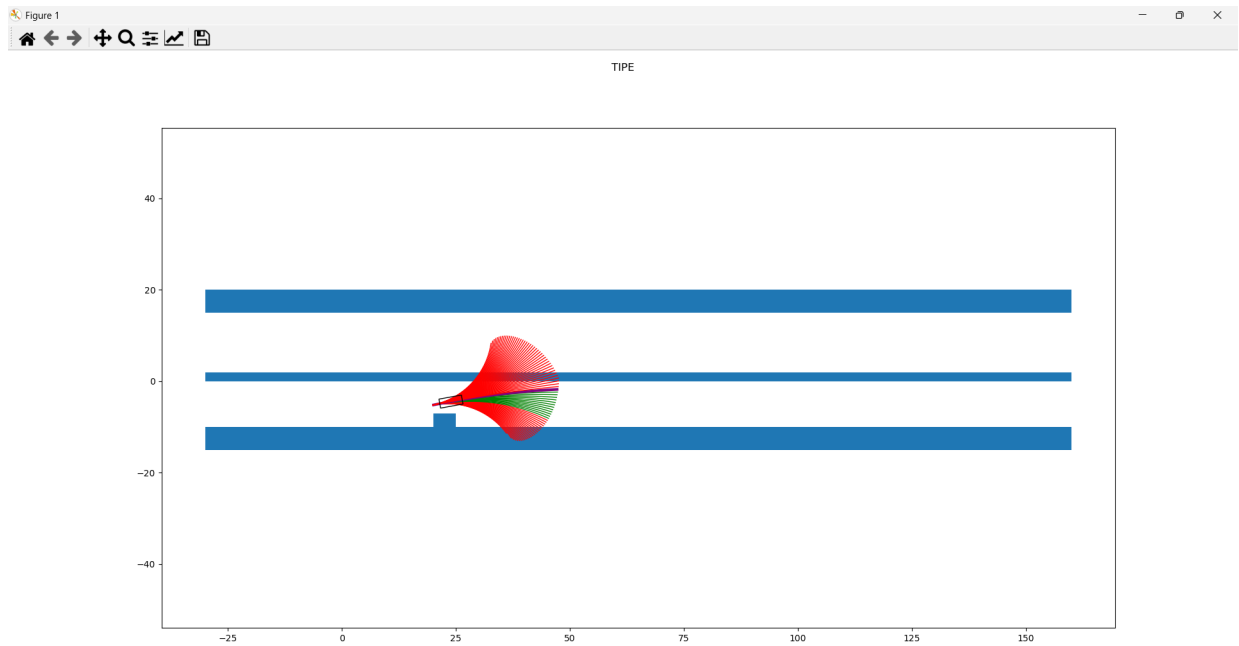
```

```

11      # TODO moins important car nécessite GPS et tt (pas forcément notre objet d'étude) :
12      ↪ faire un critère de rapprochement de la trajectoire globale (p.126 section
13      ↪ 9.2.4.3)
14
15  try:
16      # On prend la 1ere tentacule lisse qu'on trouve
17      for tentacule in tentacules_navigables:
18          if tentacule.lisse is True:
19              return tentacule
20          # Si aucune n'est lisse, on prend la dernière
21      return tentacules_navigables[-1]
22  except IndexError:
23      raise "AUCUNE TENTACULE NAVIGABLE"

```

CRITÈRE DE CHOIX D'UNE TENTACULE



```

1  """
2  Déplace la voiture de manière optimal sur un parcours.
3  """
4  class Tentacule:
5      """
6      Objet tentacule, dépendant d'une voiture, et déterminé par son ordre et son angle de
↪   braquage.
7      """
8
9      def __init__(self, voiture, k, delta_k, axes):
10         self.V_lisse = abs(self.braquage - self.voiture.braquage) / (2 *
↪         self.voiture.braquage_max)
11

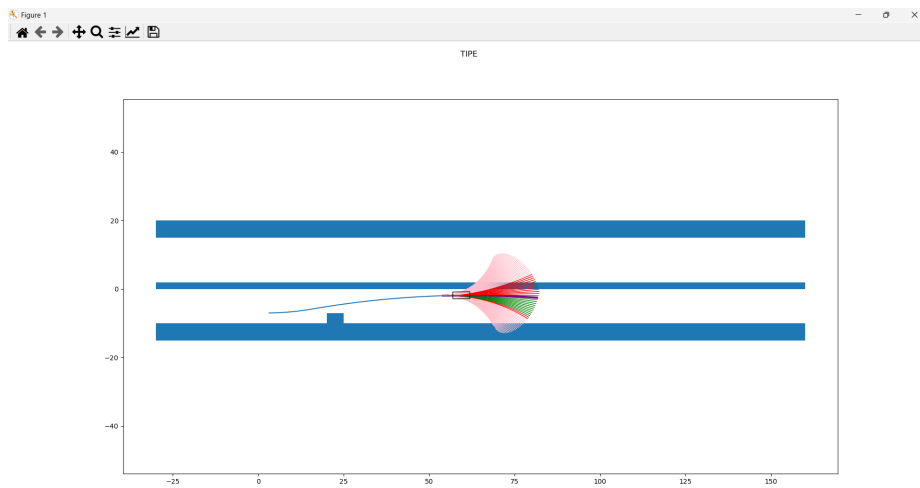
```

```

12
13 class Voiture:
14     """
15     Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
16     """
17     def choix_tentacule(self, tentacules_navigables):
18         # TODO : faire un critère de dégagement du tentacule (p. 125 section 9.2.4.1)
19         # TODO moins important car nécessite GPS et tt (pas forcément notre objet d'étude) :
↪ faire un critère de rapprochement de la trajectoire globale (p.126 section
↪ 9.2.4.3)
20
21     try:
22         return min(tentacules_navigables, key=lambda x: x.V_lisse**4 + x.braquage**2) #
↪ On trie selon une loi fonction de V_lisse (écart au braquage actuel) et de
↪ braquage (trajectoire droite). Puis on prend le min
23     except ValueError:
24         print("AUCUNE TENTACULE NAVIGABLE")
25

```

OPTIMISATION DE L'ALGORITHME ET AJOUT DE "L'HOUCHE" DE LA "VOITURE"



```

1  """
2  Déplacement de la voiture sur un parcours, en n'évaluant pas toutes les tentacules (gain de
   ↪ temps) et en affichant la trace de la voiture.
3  """
4
5
6  class Voiture:
7      """
8      Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
   ↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
9      """
10     def deplace(self):
11         """
12         S'il existe un chemin (suite de points), déplace la voiture suivant ce chemin.
13
14         Raises

```

```

15         -----
16         erreur
17             Le chemin à suivre n'est pas défini.
18
19         Returns
20         -----
21         None.
22
23         """
24         try:
25             new_x = self.chemin[1][0]
26             new_y = self.chemin[1][1]
27             vx = new_x - self.chemin[0][0]
28             vy = new_y - self.chemin[0][1]
29             new_angle = np.degrees(np.arctan2(vy, vx))
30             self.shape_voiture.set(x=new_x - self.largeur/2, y=new_y - self.longueur/2,
31                                     ↪ angle=new_angle)
32             self.x, self.y, self.angle = new_x, new_y, new_angle
33             self.chemin = np.delete(self.chemin, 0, 0)
34             if self.traceur is True:
35                 self.trace.set_data(np.append(self.trace.get_data()[0], new_x),
36                                     ↪ np.append(self.trace.get_data()[1], new_y))
37         except IndexError:
38             print('TENTACULE ÉPUISÉE, ON NE SAIT PLUS QUEL CHEMIN SUIVRE')
39
40 class Simulation_Ref_Terre:
41     """
42     Classe qui gère tout l'aspect simulation : animation + gestion des objets.
43
44     Parameters
45     -----
46     voitures: list
47         Liste des voitures présentes
48     obstacles: list # TODO
49         liste des obstacles FIXES. Ils ne seront pas animés
50     obstacles_mvt: list # TODO
51         liste des obstacles à déplacer. Fonction développée plus tard
52     """
53     def animer(self, i):
54         """
55         Anime la figure. S'occupe de déplacer les voitures et les obstacles.
56         Note : fait appel au "task-manager des tentacules" pour savoir quoi actualiser dans
57         ↪ la figure (blit)
58
59         Parameters
60         -----
61         i : int
62             Numéro d'appel de la fonction (elle est censée être appelée itérativement par
63             ↪ une matplotlib.animation.
64
65         Returns
66         -----
67         list of artists
68             Liste des dessins à actualiser (blit).
69
70         """
71         traces = []
72         # Déplace les voitures
73         for voiture in self.voitures:
74             voiture.deplace()
75         # Déplace les obstacles
76         for obstacle in self.obstacles_mvt:

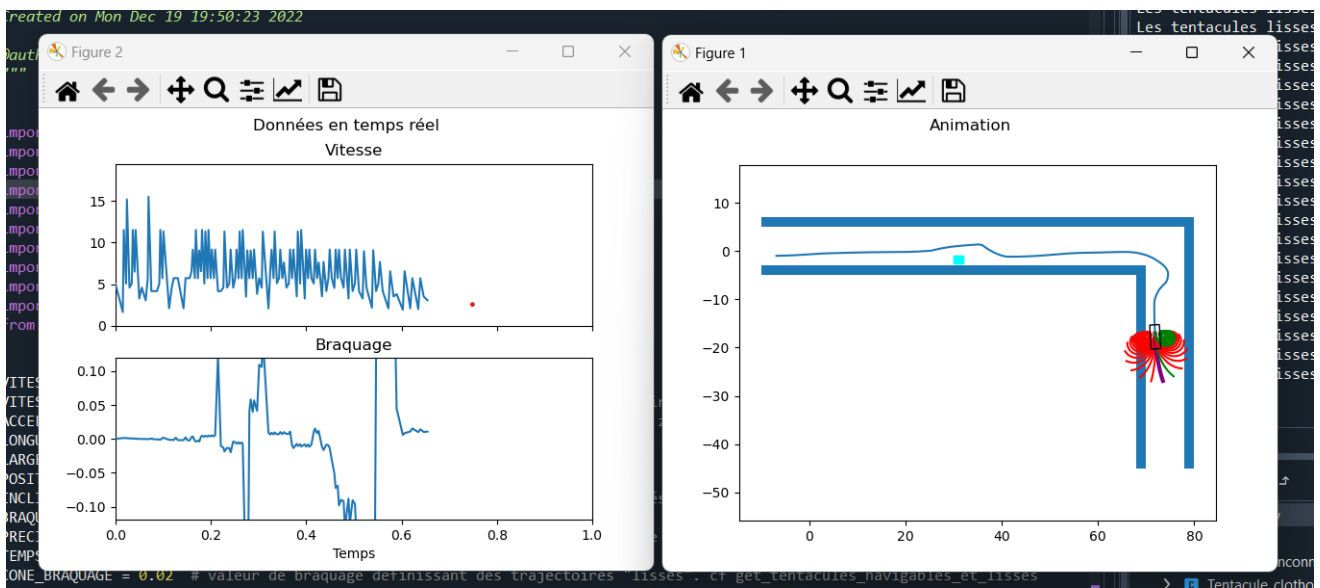
```

```

73     obstacle.deplace()
74     try:
75         # Retourne la liste des dessins à actualiser dans la figure
76         # TODO : ajouter les obstacles lors de l'import de la fonctionnalité
77         return [voiture.shape_voiture for voiture in self.voitures] + [tentacule.patch
78         ↪ for tentacule in self.new_tentacules] + [tentacule.patch for tentacule in
79         ↪ self.old_tentacules] + [obstacle.patch for obstacle in self.obstacles_mvt] +
80         ↪ [voiture.trace for voiture in self.voitures]
81     finally:
82         # Met juste après les valeurs adéquates à jour pour ne plus avoir à redessiner
83         ↪ des dessins fixes (blit)
84         self.old_tentacules = [] # Suppression des tentacules obsoletes, qu'on a bien
85         ↪ effacé avec le return
86         for tentac in self.old_tentacules:
87             tentac.patch.remove()
88         for tentac in self.new_tentacules:
89             tentac.patch.set_animated(False) # Dés-anime les nouvelles tentacules
90             ↪ (affichées avec le return) : elles ne doivent plus bouger jusqu'à une
91             ↪ redéfinition de celles-ci. Sert ?

```

AMÉLIORATIONS - CODE FINAL



```

1  # -*- coding: utf-8 -*-
2  """
3  Created on Mon Dec 19 19:50:23 2022
4
5  @author: adaml
6  """
7
8
9  import numpy as np
10 import matplotlib.pyplot as plt
11 import matplotlib.animation
12 import matplotlib.patches
13 import matplotlib.transforms
14 import matplotlib.lines
15 import scipy.interpolate

```

```

16 import shapely
17 import shapely.geometry
18 import time
19 from scipy.integrate import quad as integrale
20
21
22 VITESSE_INIT_VOIT = 5 # vitesse initiale du véhicule
23 VITESSE_MAX_VOIT = 15 # vitesse maximale (pour rester dans les zones linéaires des modèles)
24 ACCEL_LAT_MAX = 1.5 # accélération latérale maximale (pour rester dans les zones linéaires
↳ des modèles)
25 LONGUEUR_VEHIC = 5 # longueur du véhicule
26 LARGEUR_VEHIC = 2 # largeur du véhicule
27 POSITION_VOITURE_INIT = (-10, -2) # position (x, y) initiale de la voiture
28 INCLINAISON_VOITURE_INIT = 0 # inclinaison initiale de la voiture (en degrés)
29 BRAQUAGE_VOLANT_INIT = 0 # angle volant/voiture en degrés
30 PRECISION = 15 # nombre de points définissant l'arc de cercle. ie nombre de positions où la
↳ voiture ira
31 TEMPS_REAC = 90 # temps de génération de tentacules
32 CONE_BRAQUAGE = 0.02 # valeur de braquage définissant des trajectoires "lisses". cf
↳ get_tentacules_navigables_et_lisses
33 COMMENTAIRES = True # si des commentaires sur l'évolution de l'algo doivent être affichés
↳ dans la console
34 PETIT_POUCE = True # si on veut récupérer la trajectoire de la voiture
35 SECURITE = True # vérifie les collisions. False si aucune sécu, None si check sans arrêter,
↳ True si arrêt lorsque collision
36
37 # courbure = ro
38 # courbure = 1/rayon dans le cas d'un cercle
39 # braquage = delta = arctan(L/rayon) quasi logique (schéma + perpendiculaire)
40 # longueur = angle*rayon
41 # empattement +/- longueur voiture
42
43
44 class _Tentacule():
45     """
46     Classe dont héritent tout type de tentacule. Elle rassemble des méthodes communes
↳ universelles. N'est pas censée être utilisée directement ; elle est appelée
↳ automatiquement par d'autres instances.
47     """
48
49     def __init__(self, axes, voiture):
50         """
51         Parameters
52         -----
53         axes : matplotlib.axes._axes.Axes
54             Axes dans lesquels la tentacule sera affichée.
55         voiture : __main__.Voiture
56             Voiture à laquelle appartient la tentacule.
57
58         Returns
59         -----
60         None.
61
62         """
63         self.axes = axes
64         self.voiture = voiture
65         self.precision = PRECISION
66
67     def get_zone_support(self):
68         """
69         Génère et stocke une zone de support pour la tentacule (distance de sécurité autour
↳ de la tentacule pour éviter quelconque collision).

```

```

70         cf von Hundelshausen et al 2008 p.647
71
72     Raises
73     -----
74     ValueError
75         Si la vitesse du véhicule est trop grande (55km/h).
76
77     Returns
78     -----
79     shapely.geometry.polygon.Polygon
80         La zone de support.
81
82     """
83     if self.voiture.vitesse < 3:
84         self.dist_zone_support = 1.4 + 0.2*self.voiture.vitesse/3 # formule empirique
85         ↪ p.123 eq(9.6) thèse
86     elif 3 <= self.voiture.vitesse <= 15:
87         self.dist_zone_support = 1.6 + 0.6 * (self.voiture.vitesse-3) / 15 # idem
88     else:
89         raise ValueError("La vitesse est trop élevée !")
90     self.zone_support = shapely.geometry.LineString(
91         ↪ self.get_parametric()).buffer(self.dist_zone_support)
92     return self.zone_support
93
94 def is_navigable(self, objets_alentours):
95     """
96     Navigabilité de la tentacule.
97
98     Parameters
99     -----
100     objets_alentours : __main__.Voiture ou __main__.Obstacle
101         Objet ayant un attribut 'shapely_poly'.
102
103     Returns
104     -----
105     bool
106         True si la tentacule est navigable, False sinon.
107
108     """
109     self.zone_to_be_clean =
110     ↪ self.get_zone_support().intersection(self.voiture.get_espace_collision())
111     for objet in objets_alentours:
112         if objet == self.voiture: # Pas grave si la tentacule cogne sa propre voiture
113             pass
114         elif self.zone_to_be_clean.intersection(objet.shapely_poly).is_empty is False:
115             self.patch.set_color("red")
116             return False
117     return True
118
119 class Tentacule_inconnue(_Tentacule):
120     """
121     Tentacule space inconnue au bataillon. Fonctionnalités pas nécessaire pour notre étude
122     ↪ donc classe aux fonctionnalités pas totalement développées.
123     """
124
125     def __init__(self, axes, voiture, patch, braquages):
126         """
127         Parameters
128         -----
129         axes : matplotlib.axes._axes.Axes
130             Axes dans lesquels la tentacule sera affichée.

```

```

128     voiture : __main__.Voiture
129         Voiture à laquelle appartient la tentacule.
130     patch : matplotlib.patches.Patch
131         Forme de la tentacule (étant donné que sa caractérisation n'est pas explicite.
132     braquages : list ou itérable
133         Itérable contenant les braquages successifs que prendra la voiture.
134
135     Returns
136     -----
137     None.
138
139     """
140     super().__init__(axes, voiture)
141     self.patch = patch
142     self.braquages = braquages
143
144 def get_parametric(self):
145     """
146     Permet d'obtenir la forme paramétrée de la tentacule. Utilise une interpolation.
147
148     Returns
149     -----
150     chemin : numpy.ndarray
151         Abscisses et ordonnées de la tentacule.
152         Ex: numpy.array([[ x1,   y1],
153                        [ x2,   y2],
154                        [ x3,   y3]])
155
156     """
157     self.chemin = (self.patch.get_transform() - axes_simu.transData).transform_path(
158         ↪ self.patch.get_path()).cleaned().vertices[:-1][: :self.signe]
159     # Interpolation du chemin
160     self.chemin_interpol = scipy.interpolate.interp1d(self.chemin[:, 0], self.chemin[:,
161         ↪ 1])
162     lch = len(self.chemin[:, 0])
163     # Interpolation de x(t) pour faire une génération d'abscisses supplémentaires
164     points = scipy.interpolate.interp1d(range(0, (lch)*self.precision, self.precision),
165         ↪ self.chemin[:, 0]) # va jusque (lch-1)*precision inclus
166     # Génération des abscisses supplémentaires avec le points(range) et des ordonnées
167     ↪ interpolées correspondantes avec le chemin_interpol(x)
168     chemin = np.array([[x, self.chemin_interpol(x)] for x in points(range(0,
169         ↪ (lch-1)*self.precision, lch-1))])
170     return chemin
171
172 def __iter__(self):
173     """
174     Implémente l'itérabilité.
175     """
176     self.chemin = self.get_parametric()
177     return self
178
179 def __next__(self):
180     """
181     Passe à l'élément suivant (la tentacule étant un itérable) : met à jour la forme
182     ↪ paramétrée, et retourne les nouveaux états de la voiture.
183
184     Raises
185     -----
186     StopIteration
187         Si la tentacule est parcourue en entier.
188
189     Returns
190     """

```

```

184         -----
185         new_x : float
186             Nouvelle abscisse.
187         new_y : float
188             Nouvelle ordonnée.
189         dx : float
190             Déplacement effectué en x.
191         dy : float
192             Déplacement effectué en y.
193         new_angle : numpy.float64
194             Nouvel angle de la voiture.
195         braquage : numpy.float64 ou celui de la liste de braquage passée en paramètre lors
↪ de la définition de la tentacule.
196             Nouveau braquage de la voiture.
197
198         """
199         try:
200             new_x = self.chemin[1][0]
201             new_y = self.chemin[1][1]
202             dx = new_x - self.chemin[0][0]
203             dy = new_y - self.chemin[0][1]
204             new_angle = np.degrees(np.arctan2(dy, dx))
205             braquage = next(self.braquages)
206             self.chemin = np.delete(self.chemin, 0, 0)
207             return new_x, new_y, dx, dy, new_angle, braquage
208         except IndexError:
209             raise StopIteration("Fin de la tentacule")
210
211
212 class Tentacule_clothoïdaire(_Tentacule):
213     """
214     Tentacule en forme de clothoïde, dépendante d'une voiture et caractérisée par une
↪ courbure initiale et d'une variation de courbure.
215     """
216
217     def __init__(self, voiture, courbure_init, var_courbure, axes):
218         """
219         Parameters
220         -----
221         voiture : __main__.Voiture
222             Voiture à laquelle appartient la tentacule.
223         courbure_init : float
224             Courbure initiale de la tentacule.
225         var_courbure : float
226             Variation de la courbure. # TODO: clarifier avec Adrien
227         axes : matplotlib.axes._axes.Axes
228             Axes dans lesquels la tentacule sera affichée.
229
230         Returns
231         -----
232         None.
233
234         """
235         super().__init__(axes, voiture)
236
237         self.longueur = 7*self.voiture.vitesse - 5 # formule empirique, équation (9.15)
↪ p.128
238         self.courbure_init = courbure_init
239         self.var_courbure = var_courbure
240         self.courb_finale = self.courbure_init + self.var_courbure*self.longueur
241

```

```

242     self.xdata = [self.x(t, self.var_courbure, self.courb_finale) for t in
    ↪ np.linspace(0, int(self.longueur), self.precision)]
243     self.ydata = [self.y(t, self.var_courbure, self.courb_finale) for t in
    ↪ np.linspace(0, int(self.longueur), self.precision)]
244     self.patch = matplotlib.lines.Line2D(self.xdata, self.ydata, color='pink')
245     self.patch.set_transform(matplotlib.transforms.Affine2D().rotate_deg_around(0, 0,
    ↪ voiture.angle) + matplotlib.transforms.Affine2D().translate(voiture.x,
    ↪ voiture.y) + self.axes.transData)

246
247     self.abscisse_curviligne_carre = 0
248     self.braquage = 0
249     self.V_lisse = 0
250
251     def x(self, t, var_courbure, courb_finale):
252         """
253         Abscisse à t de la clothoïde : intégrale de Fresnel.
254
255         Parameters
256         -----
257         t : numpy.float64
258             Borne de l'intégrale.
259         var_courbure : float
260             Variation de la courbure.
261         courb_finale : float
262             Courbure finale de la clothoïde.
263
264         Returns
265         -----
266         None.
267
268         """
269         return integrale(lambda s: np.cos(var_courbure*s**2 + courb_finale*s)/3, 0, t)[0] #
    ↪ Intégrale de Fresnel avec facteur 1/3 correctif (meilleure mise à l'échelle)
270
271     def y(self, t, var_courbure, courb_finale):
272         """
273         Ordonnée à t de la clothoïde : intégrale de Fresnel.
274
275         Parameters
276         -----
277         t : numpy.float64
278             Borne de l'intégrale.
279         var_courbure : float
280             Variation de la courbure.
281         courb_finale : float
282             Courbure finale de la clothoïde.
283
284         Returns
285         -----
286         None.
287
288         """
289         return integrale(lambda s: np.sin(var_courbure*s**2 + courb_finale*s)/3, 0, t)[0] #
    ↪ Intégrale de Fresnel avec facteur 1/3 correctif (meilleure mise à l'échelle)
290
291     def get_parametric(self):
292         """
293         Permet d'obtenir la forme paramétrée de la tentacule. Récupère simplement les points
    ↪ car clothoïde paramétrée par définition puis transformée dans le plan.
294
295         Returns
296         -----

```

```

297     chemin : numpy.ndarray
298         Abscisses et ordonnées de la tentacule.
299         Ex: numpy.array([[ x1,   y1],
300                          [ x2,   y2],
301                          [ x3,   y3]])
302
303     """
304     chemin = (self.patch.get_transform() - axes_simu.transData).transform_path(
305         ↪ self.patch.get_path()).cleaned().vertices[:-1]
306     return chemin
307
308 def __iter__(self):
309     """
310     Implémente l'itérabilité.
311     """
312     self.chemin = self.get_parametric()
313     return self
314
315 def __next__(self):
316     """
317     Passe à l'élément suivant (la tentacule étant un itérable) : met à jour la forme
318     ↪ paramétrée, et retourne les nouveaux états de la voiture.
319
320     Raises
321     -----
322     StopIteration
323         Si la tentacule est parcourue en entier.
324
325     Returns
326     -----
327     new_x : float
328         Nouvelle abscisse.
329     new_y : float
330         Nouvelle ordonnée.
331     dx : float
332         Déplacement effectué en x.
333     dy : float
334         Déplacement effectué en y.
335     new_angle : numpy.float64
336         Nouvel angle de la voiture.
337     braquage : numpy.float64 ou celui de la liste de braquage passée en paramètre lors
338     ↪ de la définition de la tentacule.
339     Nouveau braquage de la voiture.
340
341     """
342     try:
343         new_x = self.chemin[1][0]
344         new_y = self.chemin[1][1]
345         dx = new_x - self.chemin[0][0]
346         dy = new_y - self.chemin[0][1]
347         new_angle = np.degrees(np.arctan2(dy, dx))
348         self.abscisse_curviligne_carre += new_x**2 + new_y**2
349         braquage = self.var_courbure * np.sqrt(self.abscisse_curviligne_carre)
350         self.braquage = braquage
351         self.chemin = np.delete(self.chemin, 0, 0)
352         return new_x, new_y, dx, dy, new_angle, braquage
353     except IndexError:
354         raise StopIteration("Fin de la tentacule")
355
356 class Tentacule_Circulaire(_Tentacule):
357     """

```

```

356     Objet tentacule en forme de cercle, dépendant d'une voiture, et caractérisé par son
↪ ordre et son angle de braquage.
357     """
358
359     def __init__(self, voiture, k, delta_k, axes):
360         super().__init__(axes, voiture)
361
362         self.numero = k
363         self.braquage = delta_k
364
365         self.L1 = 8 + 72 * (self.voiture.vitesse/self.voiture.vitesse_max)**1.2 # formule
↪ empirique, équation (9.5) p.123
366         # formules pour la longueur p.123 équation (9.4)
367         # formules pour le rayon de courbure p.122 équation (9.3)
368
369         if 1 <= k <= 40: # tentacule à orienter vers la droite
370             self.longueur = self.L1 + 20 * np.sqrt((k-1) / 40)
371             self.courbure = -np.tan(delta_k) / self.voiture.longueur
372
373             self.rayon = 1 / self.courbure
374             self.angle_rad = self.longueur / self.rayon
375             self.angle = self.angle_rad * 180 / np.pi
376
377             # generation de l'arc de cercle
378             self.angle_debut = 180 - self.angle
379             self.angle_fin = 180
380             centre_x = self.voiture.x + self.rayon / 2
381
382             self.signe = -1 # parcours de la tentacule dans le sens inverse, voir
↪ Voiture.update_tentacule()
383
384         elif k == 41:
385             self.longueur = self.L1 + 20 * np.sqrt((k-1) / 40)
386             self.courbure = -np.tan(delta_k) / self.voiture.longueur
387
388             self.rayon = 1 / self.courbure
389             self.angle_rad = self.longueur / self.rayon
390             self.angle = self.angle_rad * 180 / np.pi
391
392             self.angle_debut = 180
393             self.angle_fin = 180 - self.angle
394             centre_x = self.voiture.x + self.rayon / 2
395
396             self.signe = 1 # parcours de la tentacule dans le bon sens, voir
↪ Voiture.update_tentacule()
397
398         else: # 42<=k<=81 # tentacule à orienter vers la gauche
399             self.longueur = self.L1 + 20 * np.sqrt((81-k) / 40)
400             self.courbure = np.tan(delta_k) / voiture.longueur
401
402             self.rayon = 1 / self.courbure
403             self.angle_rad = self.longueur / self.rayon # en radian
404             self.angle = self.angle_rad * 180 / np.pi # en degrés
405
406             self.angle_debut = 0
407             self.angle_fin = self.angle
408             centre_x = self.voiture.x - self.rayon / 2
409
410             self.signe = 1 # parcours de la tentacule dans le bon sens, voir
↪ Voiture.update_tentacule()
411
412         # On créé le dessin de la tentacule comme si la voiture était orientée vers le Nord

```

```

413     self.patch = matplotlib.patches.Arc((centre_x, self.voiture.y), self.rayon,
    ↪     self.rayon, theta1=self.angle_debut, theta2=self.angle_fin, color='pink')
414     # On applique la rotation de la voiture au dessin de la tentacule
415     self.patch.set_transform(matplotlib.transforms.Affine2D().rotate_deg_around(
    ↪     self.voiture.x, self.voiture.y, self.voiture.angle - 90) + self.axes.transData)
416
417     self.V_lisse = abs(self.braquage - self.voiture.braquage) / (2 *
    ↪     self.voiture.braquage_max)
418
419     def get_parametric(self):
420         """
421         Permet d'obtenir la forme paramétrée de la tentacule. Modifie le patch en
    ↪     définissant un nombre de points explicite, et récupère les coordonnées.
422
423         Returns
424         -----
425         chemin : numpy.ndarray
426             Abscisses et ordonnées de la tentacule.
427             Ex: numpy.array([[ x1,   y1],
428                             [ x2,   y2],
429                             [ x3,   y3]])
430
431         """
432         # Définition d'un arc avec un nombre de points contrôlé
433         self.patch._path = matplotlib.patches.Path.arc(self.angle_debut, self.angle_fin,
    ↪         self.precision)
434         # Paramétrisation de la tentacule
435         chemin = (self.patch.get_transform() - axes_simu.transData).transform_path(
    ↪         self.patch.get_path()).cleaned().vertices[:-1][: :self.signe]
436         return chemin
437
438     def __iter__(self):
439         """
440         Implémente l'itérabilité.
441         """
442         self.chemin = self.get_parametric()
443         return self
444
445     def __next__(self):
446         """
447         Passe à l'élément suivant (la tentacule étant un itérable) : met à jour la forme
    ↪     paramétrée, et retourne les nouveaux états de la voiture.
448
449         Raises
450         -----
451         StopIteration
452             Si la tentacule est parcourue en entier.
453
454         Returns
455         -----
456         new_x : float
457             Nouvelle abscisse.
458         new_y : float
459             Nouvelle ordonnée.
460         dx : float
461             Déplacement effectué en x.
462         dy : float
463             Déplacement effectué en y.
464         new_angle : numpy.float64
465             Nouvel angle de la voiture.
466         braquage : numpy.float64 ou celui de la liste de braquage passée en paramètre lors
    ↪     de la définition de la tentacule.

```

```

467         Nouveau braquage de la voiture.
468
469         """
470     try:
471         new_x = self.chemin[1][0]
472         new_y = self.chemin[1][1]
473         dx = new_x - self.chemin[0][0]
474         dy = new_y - self.chemin[0][1]
475         new_angle = np.degrees(np.arctan2(dy, dx))
476         braquage = self.braquage
477         self.chemin = np.delete(self.chemin, 0, 0)
478         return new_x, new_y, dx, dy, new_angle, braquage
479     except IndexError:
480         raise StopIteration("Fin de la tentacule")
481
482
483 class Voiture:
484     """
485     Objet voiture, défini par sa forme. D'autres paramètres fixes sont définis après, on
↪ pourra les mettre en paramètre plus tard si plusieurs voitures il y a.
486     """
487
488     def __init__(self, shape_voiture, axes, type_tentac=Tentacule_Circulaire,
↪ nombre_tentac=80):
489         """
490         Parameters
491         -----
492         shape_voiture : matplotlib.patches.Rectangle
493         Forme de la voiture.
494         axes : matplotlib.axes._axes.Axes
495         Axes dans lesquels la voiture sera affichée.
496         type_tentac : __main___.Tentacule, optional
497         Le type de tentacule qui vont être générées par la voiture. The default is
↪ Tentacule_Circulaire.
498         nombre_tentac : int, optional
499         Nombre de tentacules à générer. The default is 80.
500
501         Returns
502         -----
503         None.
504
505         """
506
507         self.shape_voiture = shape_voiture
508         self.shape_voiture.rotation_point = 'center'
509
510         self.shapely_poly =
↪ shapely.geometry.Polygon((self.shape_voiture.get_transform()).transform_path(
↪ self.shape_voiture.get_path()).vertices)
511         print(self.shapely_poly, self.shape_voiture)
512
513         self.longueur = self.shape_voiture.get_height()
514         self.largeur = self.shape_voiture.get_width()
515         self.vitesse_max = VITESSE_MAX_VOIT
516         self.accel_lat_max = ACCEL_LAT_MAX
517
518         self.vitesse = VITESSE_INIT_VOIT
519         self.image_vitesse = self.vitesse
520         self.x, self.y = self.shape_voiture.get_center()
521         self.angle = INCLINAISON_VOITURE_INIT
522         self.braquage = 0
523         self.distance_collision = self.vitesse**2 / 2*self.accel_lat_max

```

```

524     self.courbure_max = self.accel_lat_max / self.vitesse ** 2 # formule p.122 équation
    ↪ (9.1)
525     self.braquage_max = np.arctan(self.longueur * self.courbure_max) # formule p.122
    ↪ équation (9.2)
526
527     self.type_tentac = type_tentac
528     self.nombre_tentac = nombre_tentac
529     self.tentacules = []
530
531     self.chemin = iter(()) # Chemin que la voiture a à suivre. Iterable d'une
    ↪ tentacule, cf update_chemin
532
533     self.tps_debut = time.monotonic()
534
535     self.axes_simu, self.axes_vitesse, self.axes_braquage = axes
536
537     self.traceur = PETIT_POUCE
538     self.trace = matplotlib.lines.Line2D([], [])
539     self.courbe_vitesse = matplotlib.lines.Line2D([time.monotonic() - self.tps_debut],
    ↪ [VITESSE_INIT_VOIT])
540     self.courbe_braquage = matplotlib.lines.Line2D([time.monotonic() - self.tps_debut],
    ↪ [BRAQUAGE_VOLANT_INIT])
541     self.axes_simu.add_line(self.trace)
542     self.axes_vitesse.add_line(self.courbe_vitesse)
543     self.axes_braquage.add_line(self.courbe_braquage)
544     self.axes_vitesse.set_ylim(0, 1.3 * self.vitesse_max)
545     self.axes_braquage.set_ylim(-self.braquage_max, self.braquage_max)
546
547     def collision(self, objet):
548         """
549         Si la voiture cogne l'objet passé en argument, la simulation est mise en pause.
550
551         Parameters
552         -----
553         objet : shapely.geometry.polygon.Polygon
554             Forme de l'objet (obstacle ou voiture) issue du module shapely.
555
556         Returns
557         -----
558         None.
559
560         """
561         if self.shapely_poly.intersects(objet) is True:
562             print("COLLISION VOITURE ", id(self))
563             if SECURITE is True:
564                 print("Arrêt de la simulation")
565                 self.simu.pause_replay()
566
567     def deplace(self):
568         """
569         S'il existe un chemin (itérable de la tentacule suivie), déplace la voiture suivant
    ↪ ce chemin et met à jour les données.
570
571         Raises
572         -----
573         erreur
574             Le chemin à suivre n'est pas défini ou est terminé.
575
576         Returns
577         -----
578         None.
579

```

```

580     """
581     try:
582         self.x, self.y, dx, dy, self.angle, self.braquage = next(self.chemin)
583         tp = time.monotonic() - self.tps_debut
584         dt = tp - self.courbe_vitesse.get_data()[0][-1]
585         tps = np.append(self.courbe_vitesse.get_data()[0], tp)
586         # La "vitesse" ne change pas car normalement le véhicule garde une vitesse à peu
        ↪ près constante. On peut obtenir (tracé dans les datas) la vitesse du
        ↪ véhicule sur la simulation,
587         # qui nous indique si ce qu'on voit est fidèle à la réalité ou non (temps de
        ↪ latence trop grands et déplacements chaotiques par accoups)

588
589         self.shape_voiture.set(x=self.x - self.largeur/2, y=self.y - self.longueur/2,
        ↪ angle=self.angle)
590         self.shapely_poly = shapely.geometry.Polygon((self.shape_voiture.get_transform()
        ↪ - self.axes_simu.transData).transform_path(
        ↪ self.shape_voiture.get_path()).vertices)

591
592         if self.traceur is True:
593             self.trace.set_data(np.append(self.trace.get_data()[0], self.x),
        ↪ np.append(self.trace.get_data()[1], self.y))
594             self.courbe_vitesse.set_data(tps, np.append(self.courbe_vitesse.get_data()[1],
        ↪ (dx**2 + dy**2)**0.5 / dt))
595             self.courbe_braquage.set_data(tps, np.append(self.courbe_braquage.get_data()[1],
        ↪ self.braquage))
596             self.axes_braquage.set_xlim(right=tp + 10)
597
598     except StopIteration:
599         print("TENTACULE ÉPUISÉE, ON NE SAIT PLUS QUEL CHEMIN SUIVRE")
600         print("Mise à jour forcée du chemin à suivre")
601         print("~ appel du gestionnaire de tentacules et coloration en marron ~")
602         self.simu.gestion_tentac(self)
603         for tentac in self.tentacules:
604             tentac.patch.set_linestyle(":")
605
606     def freiner(self):
607         """
608         Freine la voiture : divise la vitesse par 2, simulant un coup de frein (vitesse
        ↪ moyenne de 35km/h).
609         Arrête la simulation si la vitesse est <1m/s (on cale).
610
611         Returns
612         -----
613         None.
614
615         """
616         self.tentacule_suivie = self.choix_tentacule(self.tentacules) # TODO: modifier
        ↪ paramètre lambda avec critère dégagement de tentacule seulement (pas braquage)
617         self.tentacule_suivie.patch.set(color="black", linewidth=5)
618         self.chemin = iter(self.tentacule_suivie)
619         self.vitesse /= 2
620         if self.vitesse < 0.5:
621             print("Tu as calé bg")
622             self.simu.pause_replay(stop=True)
623
624     def update_chemin(self):
625         """
626         Supprime les tentacules actuelles et en génère de nouvelles, puis met à jour le
        ↪ chemin que la voiture doit emprunter.
627
628         Returns
629         -----

```

```

630     None.
631
632     """
633     self.remove_tentacules()
634     self.generation_tentacules()
635
636     tentacules_navigables =
637     ↪ self.get_tentacules_navigables_et_lisses(self.simu.obstacles_fixes +
638     ↪ self.simu.obstacles_mvt + self.simu.voitures)
639     if tentacules_navigables == []: # les tentacules du milieu ne sont pas navigables :
640     ↪ on va chercher les tentacules extrémales
641     print("Les tentacules lisses ne sont pas navigables")
642     tentacules_navigables =
643     ↪ self.get_all_tentacules_navigables(self.simu.obstacles_fixes +
644     ↪ self.simu.obstacles_mvt + self.simu.voitures)
645
646     try:
647         tentacule_a_suivre = self.choix_tentacule(tentacules_navigables)
648
649     except ValueError as err: # Pas de tentacule navigable
650         print(err)
651         print("Freinage !")
652         self.freiner()
653
654     else:
655         tentacule_a_suivre.patch.set_color("purple")
656         tentacule_a_suivre.patch.set_linewidth(3)
657         self.tentacule_suivie = tentacule_a_suivre
658         self.chemin = iter(tentacule_a_suivre)
659
660 def generation_tentacules(self):
661     """
662     Génère les 80 tentacules que la voiture aura la possibilité d'emprunter.
663
664     Returns
665     -----
666     None.
667
668     """
669     if self.type_tentac == Tentacule_Circulaire:
670         self.tentacules = []
671         deltas = np.linspace(-self.braquage_max, self.braquage_max, 80) # 80 est un
672         ↪ nombre empirique de tentacules défini dans la thèse (il est bon compromis
673         ↪ entre précision et temps de calcul)
674         for k in range(1, 81):
675             self.tentacules.append(self.type_tentac(self, k, deltas[k-1],
676             ↪ self.axes_simu)) # Ajout de la tentacule aux tentacules appartenant au
677             ↪ véhicule
678         for k in range(80):
679             self.axes_simu.add_patch(self.tentacules[k].patch) # Ajout de la tentacule
680             ↪ à la figure
681     elif self.type_tentac == Tentacule_clothoïdaire:
682         self.tentacules = []
683         ro_max = self.accel_lat_max / self.vitesse**2
684         courbure_tentac_init = self.braquage * np.pi / 180
685         rap_min = 2*(-ro_max - courbure_tentac_init) / self.distance_collision
686         rap_max = 2*(+ro_max - courbure_tentac_init) / self.distance_collision
687         deltas = np.linspace(rap_min, rap_max, 40)
688         for rap in deltas:
689             self.tentacules.append(Tentacule_clothoïdaire(self, courbure_tentac_init,
690             ↪ rap, self.axes_simu))
691     for tentacule in self.tentacules:

```

```

681         self.axes_simu.add_line(tentacule.patch)
682     else:
683         pass # TODO: compléter avec tentac inconnue
684
685 def remove_tentacules(self):
686     """
687     Enlève les tentacules (qui appartiennent au véhicule) de la figure.
688
689     Returns
690     -----
691     None.
692
693     """
694     for tentacule in self.tentacules:
695         tentacule.patch.set(visible=False) # car si simple remove, on perd les
        ↪ axes_simu de référence et c pas bon pour le blit...
696
697 def get_espace_collision(self):
698     """
699     Génère et stocke une zone de support pour le véhicule (distance de sécurité autour
    ↪ de la voiture pour éviter quelconque collision).
    cf von Hundelshausen et al 2008 p.650
700
701     Returns
702     -----
703     shapely.geometry.polygon.Polygon
704         La zone de support évitant les collisions.
705
706     """
707     self.distance_collision = self.vitesse**2 / 2*self.accel_lat_max
708     self.espace_collision = shapely.geometry.Point((self.x,
    ↪ self.y)).buffer(self.distance_collision + self.longueur/2)
709     return self.espace_collision
710
711 def is_tentacule_navigable(self, tentacule, objets_alentours):
712     """
713     Teste si la tentacule spécifiée est navigable (cogne des objets alentours dans le
    ↪ périmètre de sécurité). Méthode non utilisée.
714
715     Parameters
716     -----
717     tentacule : __main__.Tentacule
718         Tentacule à tester.
719     objets_alentours : __main__.Voiture ou __main__.Obstacle
720         Objets à éviter.
721
722     Returns
723     -----
724     bool
725         True si la tentacule est navigable, False sinon.
726
727     """
728     return tentacule.is_navigable(objets_alentours)
729
730 def choix_tentacule(self, tentacules_navigables, ponderation_meilleure_tentac=lambda
    ↪ tentac: tentac.V_lisse**4 + tentac.braquage**2):
731     """
732     Choisi une tentacule à suivre.
733
734     Parameters
735     -----
736     tentacules_navigables : list de __main__.Tentacule
737

```

```

738         Tentacules dispo à trier pour choisir.
739         ponderation_meilleure_tentac : callable, optional
740         Fonction de tri. The default is lambda tentac: tentac.V_lisse**4 +
↪ tentac.braquage**2.
741
742         Raises
743         -----
744         ValueError
745             S'il n'y a aucune tentacule navigable (tentacules_navigables = [] par exemple).
746
747         Returns
748         -----
749         __main___.Tentacule
750             Tentacule à suivre (meilleure parmi celles données en argument).
751
752         """
753         # TODO : faire un critère de dégagement du tentacule (p. 125 section 9.2.4.1)
754         # TODO moins important car nécessite GPS et tt (pas forcément notre objet d'étude) :
↪ faire un critère de rapprochement de la trajectoire globale (p.126 section
↪ 9.2.4.3)
755
756         try:
757             self.tentacule_suivie = min(tentacules_navigables,
↪ key=ponderation_meilleure_tentac) # On trie selon une loi empirique
↪ fonction de V_lisse (écart au braquage actuel) et de braquage (trajectoire
↪ droite). Puis on prend le min
758             return self.tentacule_suivie
759         except ValueError:
760             raise ValueError("AUCUNE TENTACULE NAVIGABLE")
761
762     def get_all_tentacules_navigables(self, objets_alentours):
763         """
764         Permet d'obtenir toutes les tentacules navigables.
765
766         Parameters
767         -----
768         objets_alentours : __main___.Voiture ou __main___.Obstacle
769             Objets à éviter.
770
771         Returns
772         -----
773         tentacules_navigables : list of __main___.Tentacule
774             Liste des tentacules navigables et que la voiture peut emprunter.
775
776         """
777         tentacules_navigables = []
778         for tentacule in self.tentacules:
779             if tentacule.is_navigable(objets_alentours):
780                 tentacule.patch.set_color('g')
781                 tentacules_navigables.append(tentacule)
782         return tentacules_navigables
783
784     def _simu(self, simu):
785         """
786         À ne pas appeler directement. Permet de définir la simulation dans laquelle évolue
↪ la voiture.
787
788         Parameters
789         -----
790         simu : TYPE
791             DESCRIPTION.
792

```

```

793         Returns
794         -----
795         None.
796
797         """
798         self.simu = simu
799
800     def get_tentacules_navigables_et_lisses(self, objets_alentours):
801         """
802         Permet d'obtenir toutes les tentacules navigables et 'lisses' ie dont la variation
803 ↪ de braquage avec le braquage courant demandée n'est pas trop forte (<CONE_BRAQUAGE).
804
805         Parameters
806         -----
807         objets_alentours : __main__.Voiture ou __main__.Obstacle
808             Objets à éviter.
809
810         Returns
811         -----
812         tentacules_navigables : list of __main__.Tentacule
813             Liste des tentacules navigables et que la voiture peut emprunter.
814
815         """
816         tentacules_lisses_et_navigables = []
817         for tentacule in [tentac for tentac in self.tentacules if abs(tentac.braquage -
818 ↪ self.braquage) < CONE_BRAQUAGE]:
819             if tentacule.is_navigable(objets_alentours):
820                 tentacule.patch.set_color('g')
821                 tentacules_lisses_et_navigables.append(tentacule)
822         return tentacules_lisses_et_navigables
823
824 class Obstacle():
825     """
826     Instance commune à tous type d'obstacle. Réunit des méthodes et attributs universels. Ne
827 ↪ pas appeler directement, est automatiquement appelée par d'autres instances.
828     """
829
830     def __init__(self, patch):
831         """
832         Parameters
833         -----
834         patch : matplotlib.patches.Rectangle
835             Forme de l'obstacle.
836
837         Returns
838         -----
839         None.
840
841         """
842         self.patch = patch
843         self.x, self.y = self.patch.get_center()
844         self.shapely_poly =
845 ↪ shapely.geometry.Polygon((self.patch.get_transform()).transform_path(
846 ↪ self.patch.get_path()).vertices)
847
848 class Obstacle_fixe(Obstacle):
849     """
850     Instance pour un obstacle qui ne bougera pas au cours du temps.
851     """

```

```

850 def __init__(self, patch):
851     """
852     Parameters
853     -----
854     patch : matplotlib.patches.Rectangle
855           Forme de l'obstacle.
856
857     Returns
858     -----
859     None.
860
861     """
862     super().__init__(patch)
863
864
865 class Obstacle_mvt():
866     """
867     Instance pour un obstacle qui est amené à bouger au cours du temps. Fonctionnalité
868     ↪ encore non développée correctement.
869     """
870
871     def __init__(self):
872         pass
873
874 class Simulation_Ref_Terre:
875     """
876     Classe qui gère tout l'aspect simulation : animation + gestion des objets.
877
878     Parameters
879     -----
880     voitures: list
881           Liste des voitures présentes
882     obstacles: list # TODO
883           liste des obstacles FIXES. Ils ne seront pas animés
884     obstacles_mvt: list # TODO
885           liste des obstacles à déplacer. Fonction développée plus tard
886     """
887
888     def __init__(self, voitures=[], obstacles_fixes=[], obstacles_mvt=[], animations=[],
889     ↪ timers=[]):
890         """
891         Parameters
892         -----
893         voitures : list of __main__.Voiture, optional
894               Voitures dans la simulation. The default is [].
895         obstacles_fixes : list of __main__.Obstacle_fixe, optional
896               Obstacles fixes dans la simulation. The default is [].
897         obstacles_mvt : list of __main__.Obstacle_mvt, optional
898               Obstacles en mouvement dans la simulation. The default is [].
899         animations : list of matplotlib.animation.FuncAnimation, optional
900               Animations qui gèrent l'affichage et l'interaction. The default is [].
901         timers : list of matplotlib.backends.backend_qt.TimerQT, optional
902               Timers qui gèrent l'animation et l'interaction. The default is [].
903
904     Returns
905     -----
906     None.
907
908     """
909     self.paused = False

```

```

910     self.animations = animations
911     self.timers = timers
912
913     self.voitures = voitures
914     self.obstacles_fixes = obstacles_fixes
915     self.obstacles_mvt = obstacles_mvt
916     self.new_tentacules = [] # liste des tentacules à afficher (blit)
917     self.old_tentacules = [] # liste des tentacules qui ne sont plus d'actualité, mais
    ↪ qui doivent être supprimées (blit)
918
919     for voiture in self.voitures:
920         voiture._simu(self)
921         self.gestion_tentac(voiture) # créé les tentacules de la voiture, et créé un
    ↪ chemin à suivre (surement inexistant jusqu'alors...)
922
923     self.commentaires = COMMENTAIRES
924
925     def animer(self, i):
926         """
927         Anime la figure. S'occupe de déplacer les voitures et les obstacles.
928         Note : fait appel au "task-manager des tentacules" pour savoir quoi actualiser dans
    ↪ la figure (blit)
929
930         Parameters
931         -----
932         i : int
933             Numéro d'appel de la fonction (elle est censée être appelée itérativement par
    ↪ une matplotlib.animation.
934
935         Returns
936         -----
937         list of artists
938             Liste des dessins à actualiser (blit).
939
940         """
941         # Déplace les voitures
942         for voiture in self.voitures:
943             voiture.deplace()
944         # Déplace les obstacles
945         for obstacle in self.obstacles_mvt:
946             obstacle.deplace()
947         try:
948             # Retourne la liste des dessins à actualiser dans la figure
949             # TODO : ajouter les obstacles lors de l'import de la fonctionnalité
950             return [item for item in [voiture.shape_voiture, voiture.trace,
    ↪ voiture.courbe_vitesse, voiture.courbe_braquage] for voiture in
    ↪ self.voitures] + [objet.patch for objet in self.old_tentacules +
    ↪ self.new_tentacules + self.obstacles_fixes + self.obstacles_mvt]
951         finally:
952             # Met juste après les valeurs adéquates à jour pour ne plus avoir à redessiner
    ↪ des dessins fixes (blit)
953             self.old_tentacules = [] # Suppression des tentacules obsoletes, qu'on a bien
    ↪ effacé avec le return
954             for tentac in self.old_tentacules:
955                 tentac.patch.remove()
956             for tentac in self.new_tentacules:
957                 tentac.patch.set_animated(False) # Dés-anime les nouvelles tentacules
    ↪ (affichées avec le return) : elles ne doivent plus bouger jusqu'à une
    ↪ redéfinition de celles-ci. Sert ?
958
959     def gestion_tentac(self, voiture):
960         """

```

```

961         "Task-manager des tentacules"... Gère l'ajout et la suppression des tentacules
↪ (appartenant à la voiture *voiture*) de la figure.
962
963     Parameters
964     -----
965     voiture : Voiture
966         objet voiture.
967
968     Returns
969     -----
970     None.
971
972     """
973     for tentac in voiture.tentacules: # pour mettre à jour et enlever les tentacules
↪ qui vont être supprimées
974         tentac.patch.set_animated(True) # Anime les tentacules à supprimer (elles
↪ étaient dés-animées) (blit)
975         self.old_tentacules.append(tentac) # ajout à la liste des dessins à actualiser
↪ (blit)
976
977     voiture.update_chemin() # suppression du système actuel de tentacules pour la
↪ voiture, puis génération de nouvelles tentacules pour la voiture
978
979     for tentac in voiture.tentacules: # pour mettre à jour et ajouter les tentacules
↪ qui vont être ajoutées
980         tentac.patch.set_animated(True) # anime les tentacules à ajouter (blit)
981         self.new_tentacules.append(tentac) # ajout à la liste des dessins à actualiser
↪ (blit)
982
983     def collisions(self):
984         """
985         Teste les collisions pour chaque voiture.
986
987         Returns
988         -----
989         None.
990
991         """
992         for voiture in self.voitures:
993             for obstacle in self.obstacles_fixes:
994                 voiture.collision(obstacle.shapely_poly)
995             for obstacle in self.obstacles_mvt:
996                 voiture.collision(obstacle.shapely_poly)
997             for voit in self.voitures:
998                 if voit != voiture: # pas de collision qd la voiture se touche elle-même
999                     voiture.collision(voit.shapely_poly)
1000
1001     def pause_replay(self, stop=False):
1002         """
1003         Met en pause ou remet en play.
1004
1005         Returns
1006         -----
1007         None.
1008
1009         """
1010         if self.paused:
1011             print("PLAY")
1012             for anim in self.animations:
1013                 anim.resume()
1014             for timer in self.timers:
1015                 timer.start()

```

```

1016         self.paused = False
1017     else:
1018         print("PAUSE")
1019         for anim in self.animations:
1020             anim.pause()
1021         for timer in self.timers:
1022             timer.stop()
1023         """
1024         if stop is False:
1025             self.paused = True # Si juste une pause on le fait savoir, si c'est un
↪ stop, on fait croire que c'est encore la simu (pour que qd on replay ça replay pas)
1026         """
1027         self.paused = True
1028
1029     def ajout_anim(self, anim):
1030         """
1031         Ajoute une animation.
1032
1033         Parameters
1034         -----
1035         anim : matplotlib.animation.FuncAnimation
1036             Animation à ajouter.
1037
1038         Returns
1039         -----
1040         None.
1041
1042         """
1043         self.animations.append(anim)
1044
1045     def ajout_timer(self, timer):
1046         """
1047         Ajoute un timer.
1048
1049         Parameters
1050         -----
1051         timer : matplotlib.backends.backend_qt.TimerQT
1052             Timer à ajouter.
1053
1054         Returns
1055         -----
1056         None.
1057
1058         """
1059         self.timers.append(timer)
1060
1061
1062     # création figure + axes
1063     figure_ref_terre, axes_simu = plt.subplots()
1064     figure_data, (axes_vitesse, axes_braquage) = plt.subplots(2, 1, sharex=True)
1065     # titre
1066     figure_ref_terre.suptitle("Animation")
1067     figure_data.suptitle("Données en temps réel")
1068     axes_vitesse.set_title("Vitesse")
1069     axes_braquage.set_title("Braquage")
1070     axes_braquage.set_xlabel("Temps")
1071
1072     # création de la forme de la voiture
1073     shape_voiture = matplotlib.patches.Rectangle(POSITION_VOITURE_INIT, LONGUEUR_VEHIC,
↪ LARGEUR_VEHIC, fill=False, zorder=float('inf'))
1074     shape_voiture.rotation_point = 'center'
1075     shape_voiture.set_angle(INCLINAISON_VOITURE_INIT)

```

```

1076
1077 # création de la voiture
1078 voiture = Voiture(shape_voiture, [axes_simu, axes_vitesse, axes_braquage],
1079 ↪ Tentacule_clothoïdaire)
1079
1080 # ajout du dessin de la voiture à la figure
1081 axes_simu.add_patch(voiture.shape_voiture)
1082
1083 # création de la forme de l'obstacle
1084 shape_obstacle = matplotlib.patches.Rectangle((30, -2.5), 2, 1.5, fill=True,
1085 ↪ zorder=float('inf'), color='cyan')
1086 obstacle1 = Obstacle_fixe(shape_obstacle)
1087 axes_simu.add_patch(obstacle1.patch)
1088
1089 # création des bords de route
1090 shape_route_haut = matplotlib.patches.Rectangle((-10, 5), 90, 2)
1091 route_haut = Obstacle_fixe(shape_route_haut)
1092 axes_simu.add_patch(route_haut.patch)
1093
1094 shape_route_bas = matplotlib.patches.Rectangle((-10, -5), 78, 2)
1095 route_bas = Obstacle_fixe(shape_route_bas)
1096 axes_simu.add_patch(route_bas.patch)
1097
1098 shape_route_gauche = matplotlib.patches.Rectangle((68, -45), 2, 42)
1099 route_gauche = Obstacle_fixe(shape_route_gauche)
1100 axes_simu.add_patch(route_gauche.patch)
1101
1102 shape_route_droite = matplotlib.patches.Rectangle((78, -45), 2, 50)
1103 route_droite = Obstacle_fixe(shape_route_droite)
1104 axes_simu.add_patch(route_droite.patch)
1105
1106 # création de la simulation
1107 simulation = Simulation_Ref_Terre([voiture], [obstacle1, route_bas, route_haut,
1108 ↪ route_droite, route_gauche])
1109
1110 # création et lancement de l'animation
1111 anim = matplotlib.animation.FuncAnimation(figure_ref_terre,
1112 ↪ simulation.animer,
1113 ↪ init_func=lambda: [voiture.shape_voiture for
1114 ↪ voiture in simulation.voitures] +
1115 ↪ [tentacule.patch for tentacule in
1116 ↪ simulation.new_tentacules] + [tentacule.patch
1117 ↪ for tentacule in simulation.old_tentacules],
1118 ↪ fargs=[],
1119 ↪ interval=35,
1120 ↪ cache_frame_data=False,
1121 ↪ blit=True,
1122 ↪ repeat=True)
1123
1124 # création d'un timer
1125 timer = figure_ref_terre.canvas.new_timer(interval=TEMPS_REAC)
1126 if SECURITE is not False:
1127     timer2 = figure_ref_terre.canvas.new_timer(40)
1128     timer2.add_callback(simulation.collisions)
1129     timer2.start()
1130     simulation.ajout_timer(timer2)
1131     figure_ref_terre.canvas.mpl_connect('close_event', lambda event: timer2.stop())
1132 # ajout d'une fonction appelée itérativement
1133 timer.add_callback(simulation.gestion_tentac, voiture)
1134 # démarrage du timer
1135 timer.start()
1136

```

```

1131 simulation.ajout_anim(anim)
1132 simulation.ajout_timer(timer)
1133 # stop le timer si on ferme la figure (pour éviter qu'une fonction continue de tourner en
    ↪ boucle sans intérêt une fois tout terminé, c'est plutôt mal vu...!)
1134 figure_ref_terre.canvas.mpl_connect('close_event', lambda event: timer.stop())
1135 figure_ref_terre.canvas.mpl_connect('key_press_event', lambda event: (print("Key Pressed"),
    ↪ simulation.pause_replay()))
1136
1137 axes_simu.autoscale()
1138 # axes_simu.set(xlim=(-50, 50), ylim=(-50, 150))
1139 axes_simu.axis('equal')
1140 plt.show()
1141

```